Chapter 3

Hashing

References:

1. R.Sedgewick and K. Wayne, Algorithms (4th Ed.), Addison-Wesley.
2. E. Horowitz, S. Sahni, S. Anderson-Freed, Fundamentals of Data Structures in C, Silicon Press, 2nd Edition.

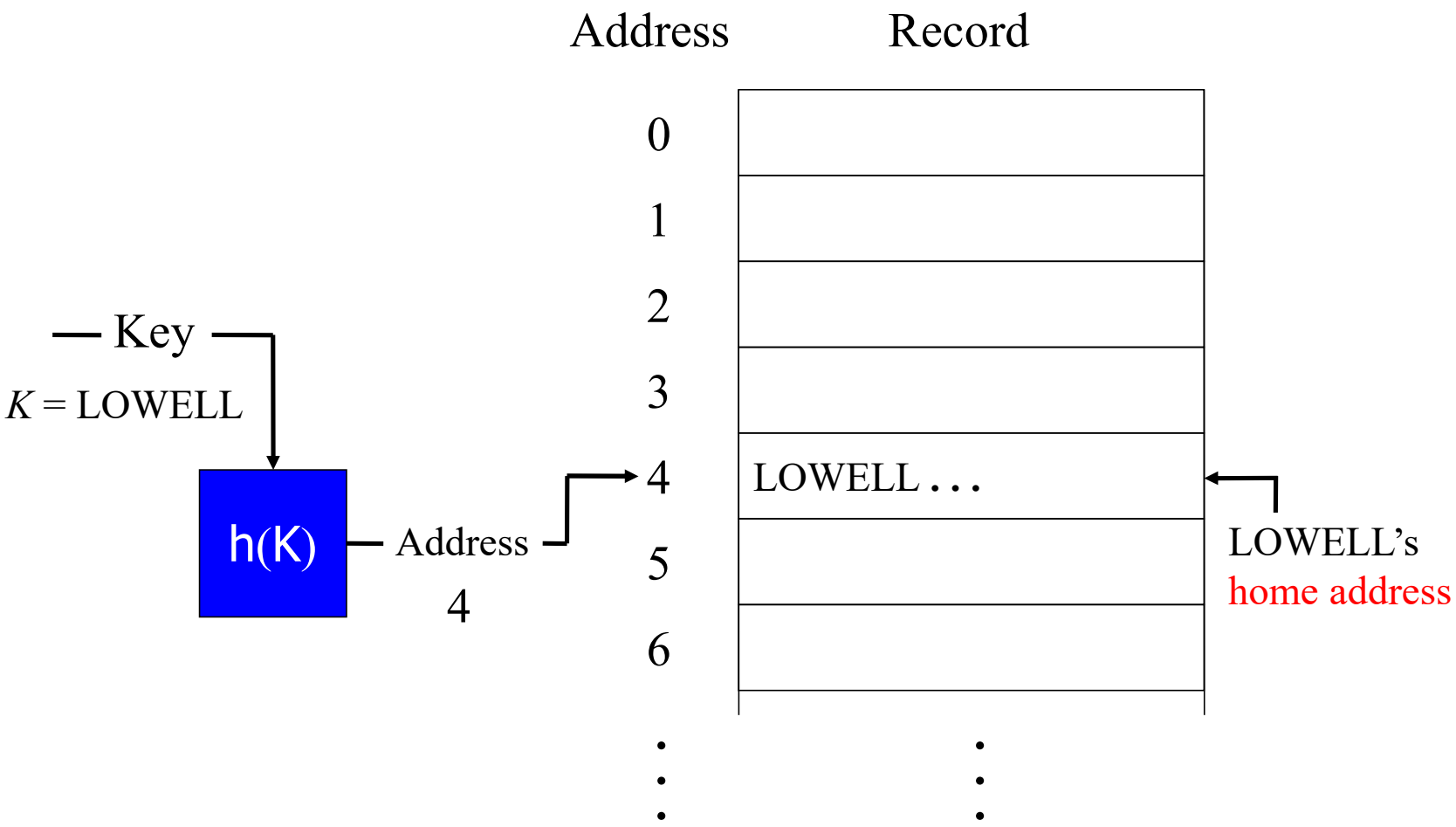
목차

해싱의 정의
해시 함수(Hash Functions)
Separate Chaining
선형 탐색(Linear Probing)
Other Collision Resolutions Techniques
동적 해싱(Dynamic Hashing)

1. 해싱의 정의

- Hashing의 목적
  - O(1) access를 지원
  - 이진검색 트리: $O(\log_2 N)$

- What is Hashing?
  - K를 키로 하는 레코드를 h(K) 주소에 저장
  - h(K): hash function
    - 주소는 h(K) 함수에 의해 무작위로 선정
    - $h(K_1) = h(K_2)$, even though $K_1 \neq K_2$ ← Collision


Address
Record
0
1
2
3
4
5
6
LOWELL . . .
Key
K = LOWELL
h(K)
Address
4
LOWELL's
home address

충돌(Collision)

- Collision
  - Synonyms: $K_1 \neq K_2$ such that $h(K_1) = h(K_2)$
  - 해결방법
    - Perfect hashing algorithm 사용
    - Collision이 적게 발생하는 hash function 구현
    - Collision 발생 시, 이를 처리하는 알고리즘 개발

- 해싱의 두 가지 요소
  - 해시 함수를 설계
  - 충돌 해결 프로세스를 설계
    - Separate chaining
    - Linear probing

2. 해시 함수: h

- 역할: h(key) → 테이블 주소
  - 계산의 효율성
  - 키들을 테이블에 균등히 배분

- 예: (전화번호, 이름) 쌍을 1000개의 테이블에 저장
  - 함수 1: 전화번호 앞의 세 자리를 주소로 반환
  - 함수 2: 전화번호 뒤의 세 자리를 주소로 반환
  - 다른 방법?

- 예: (주민등록번호, 이름) 쌍을 1000개의 테이블에 저장
  - 함수 1: 주민등록번호 앞의 세 자리 이용
  - 함수 2: 주민등록번호 뒤의 세 자리 이용
  - 다른 방법?

2.1 Java의 해시 함수

- 모든 Java 클래스들은 hashCode() 함수를 구현
  - 32-bit 정수를 반환
  - 요구 사항: x.equal(y) → (x.hashCode() == y.hashCode())
  - 희망 사항: !x.equals(y) → (x.hashCode() != y.hashCode())

```
public final class Integer {
   private final int value;
   ...
   public int hashCode()
   { return value; }
}
```

```
public final class Boolean {
   private final boolean value;
   ...
   public int hashCode() {
      if (value) return 1231;
      else return 1237;
   }
}
```

```
public final class Double {
   private final double value;
   ...
   public int hashCode() {
      long bits = doubleToLongBits(value);
      return (int) (bits ^ (bits >> 32));
   }
}
```

64비트 long으로 변경 후,
상위 32비트와 하위 32비트를
exclusive-or

Java String의 해시 함수

```
public final class String
{
   private int hash = 0;
   private final char[] s;
   ...
   public int hashCode() {
      int h = hash;
      if (h != 0) return h;
      for (int i = 0; i < length(); i++)
         h = s[i] + (31 * h);
      hash = h;
      return h;
   }
}
```

```
예.  String s = "call";
     int code = s.hashCode();

108 + 31*(108 + 31*(97 + 31*(99)))
= 3045982
```

- 문자의 Unicode를 이용
- 문자열의 모든 문자들을 이용하여 해시 함수를 구성
  - 31과 같이 작은 소수를 이용
  - Honor's method
- 복잡한 해쉬 함수가 여러 번 실행되는 것을 방지
  - 해시 함수의 값을 저장
  - 한번만 해시 함수를 실행하고, 이후의 경우에는 저장된 값을 반환

사용자 클래스의 해시 함수

```
public class Transaction {
  private final String who;
  private final Date when;
  private final double amount;
  ...

  public int hashCode() {
    int hash = 17;
    hash = 31*hash + who.hashCode();
    hash = 31*hash + when.hashCode();
    hash = 31*hash + ((Double) amount).hashCode();
    return hash;
  }
}
```

- Java에서 지원하는 클래스의 hashCode()를 활용
- Primitive 타입은 wrapper class의 hashCode()를 활용
- Honor's method를 이용하여 모든 요소들이 해시 값에 참여할 수 있도록 할 것.

2.2 해시 함수의 설계

- 해시 코드를 만드는 표준 방법론
  - 각 필드를 31x + y 규칙을 이용하여 결합
  - 필드가 primitive type이면, wrapper type의 hashCode()를 이용
  - 필드가 null이면, 0을 반환
  - 필드가 reference type이면, hashCode() 이용
  - 필드가 배열이면, 각 원소값을 모두 적용
    - Primitive type: Arrays.hashCode()
    - Reference type: Arrays.deepHashCode()

- 가장 기본적인 규칙
  - 해시 코드를 만들기 위해서 모든 필드를 이용할 것!

해시 코드에서 해시 함수로 변환

- 해시 코드: $-2^{31} \sim 2^{31} - 1$ 사이의 정수
- 해시 함수: 0 ~ M – 1 사이의 정수 (M: 테이블의 크기)
- 해시 코드를 해시 함수로 변환하기 위해서 '% M' 필요
- 음수에 주의 (해시 코드의 연산 과정에서 overflow 발생)

```
private int hash(Key key)
{ return key.hashCode() % M; }
```

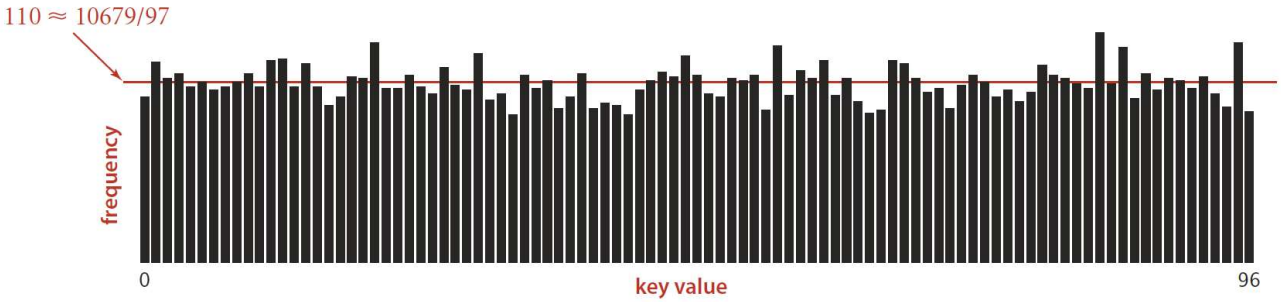
```
private int hash(Key key)
{ return (key.hashCode() & 0x7fffffff) % M; }
```

해시 함수들의 예

- Divide the key by a number
  - 가장 널리 사용
  - 특정 주소공간으로 사상하기 위하여 필수적
  - 소수로 나눌 경우, collision 발생 확률 ↓
- Square the key and take the middle
  - 예) 453 = 20**52**09 → '52'를 주소로 선택
  - overflow 에 주의
- Radix transformation
  - 예) 453_{10}을 453_{11} 로 간주

균등 해싱 가정

- 정의: 해시 함수에 의해 각 키는 0부터 M – 1 사이의 정수에 균등히 배분된다.
  - 현실적으로 이런 해시 함수를 만드는 것은 불가능
  - 그러나 적절한 해시 함수를 이용할 때 유사한 성능을 나타내는 경우가 많음
  - 입력될 Key의 성격 파악 & 실험 필요

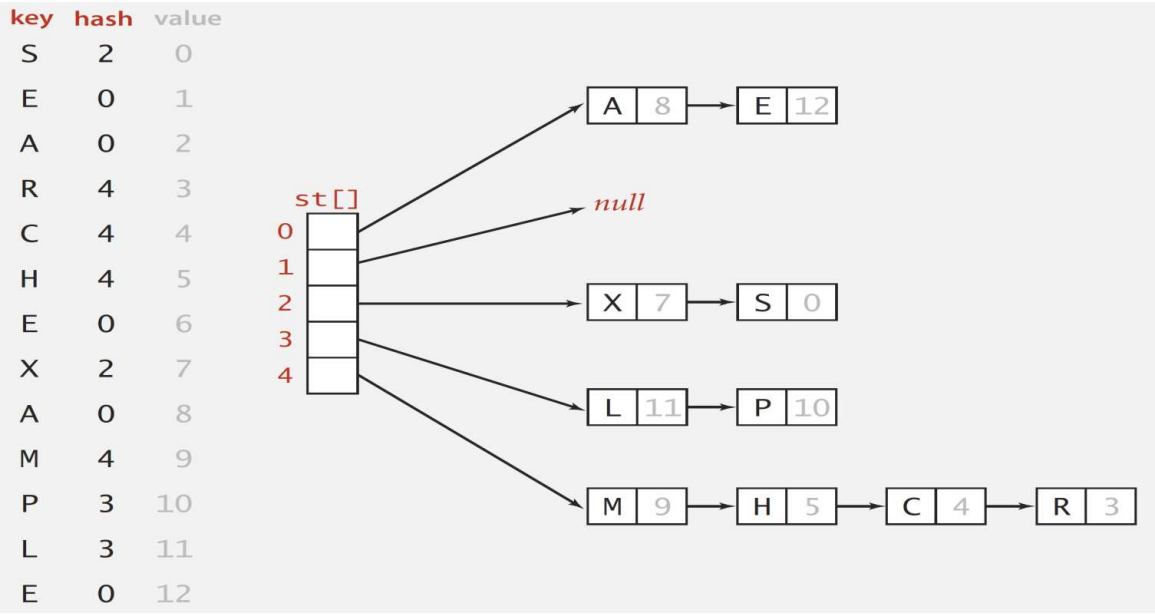


Hash value frequencies for words in *Tale of Two Cities* (10,679 keys, $M = 97$)

Java의 String을 이용하여 소설에 나오는 단어들을 어느 정도 균등하게 배분

3. Separate Chaining

- 동일한 해시 값을 갖는 원소들을 리스트로 연결
  - M: 주소 공간, N: 원소의 수 → M < N
  - h(key): 0 ~ M – 1 사이의 정수 i로 매핑
  - 삽입: i번째 리스트의 제일 앞에 추가 (존재하지 않는 키의 경우)
  - 검색: i번째 리스트만 검사



3.1 Java 구현 – SeparateChainingHashST

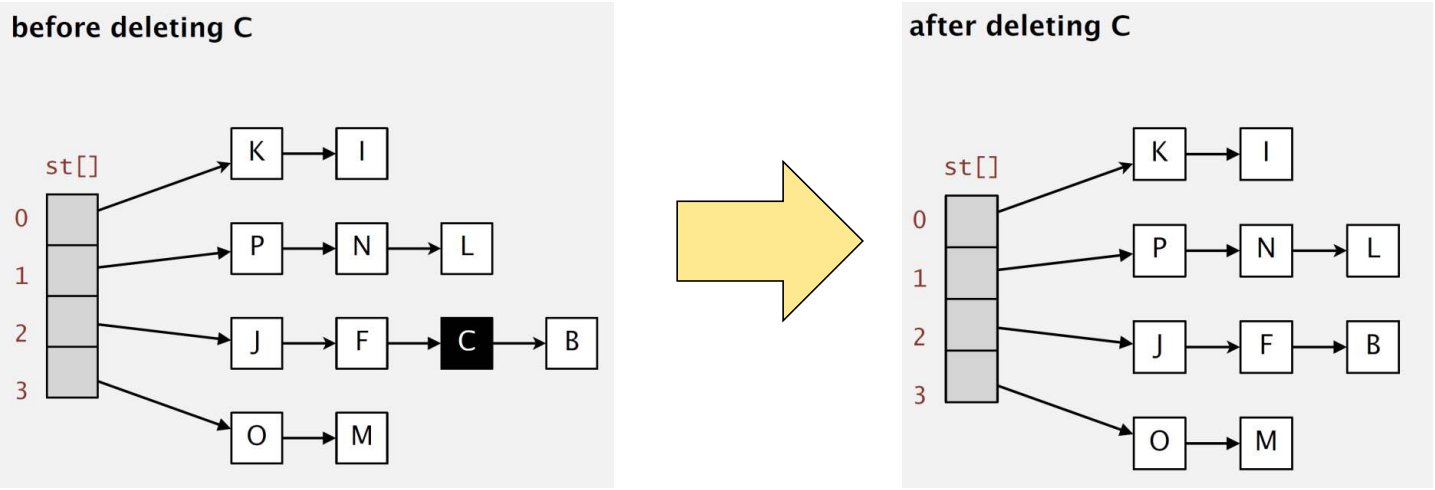
- 기본 개념
  - 동일한 해시 값을 갖는 원소들은 SequentialSearchST로 구현

- 구현 방법
  - M 개의 배열을 준비
  - 각 배열은 SequentialSearchST의 참조를 저장
  - get()과 put() 등의 함수는 SequentialSearchST의 get()과 put()을 이용하여 구현

- 순서화 연산은 지원되지 않음

삭제 연산의 구현

- 키를 포함한 리스트만 고려



SeparateChainingHashST 클래스(1)

```
public class SeparateChainingHashST<K,V> {
    private int N;      // 테이블에 입력된 원소의 수
    private int M;      // 해시 테이블의 크기
    private SequentialSearchST<K,V>[] st;    // 순차 연결 리스트의 배열로 테이블 구성

    public SeparateChainingHashST() {  this(997); }    // default: 소수
    public SeparateChainingHashST(int M) {   // M을 입력받는 생성자
        this.M = M;
        st = (SequentialSearchST<K,V>[]) new SequentialSearchST[M];
        for (int i = 0; i < M; i++)
            st[i] = new SequentialSearchST<K,V>();
    }

    public boolean contains(K key) { return get(key) != null; }
    public boolean isEmpty() { return N == 0; }
    public int size() { return N; }

    // 해시 함수: M으로 나머지 연산
    private int hash(K key) {    return (key.hashCode() & 0x7fffffff) % M;    }
```

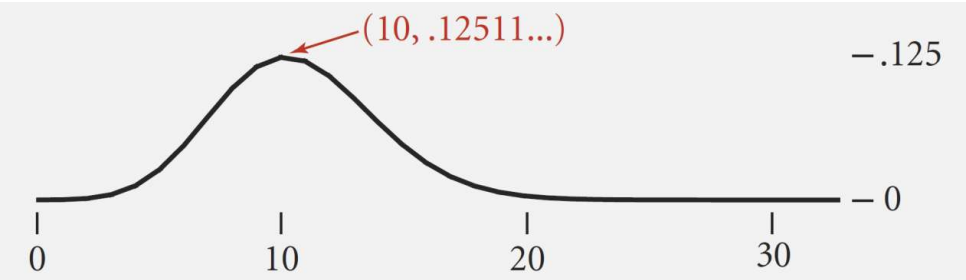
SeparateChainingHashST 클래스(2)

```
public V get(K key)   {   return st[hash(key)].get(key);    }      // 리스트를 검색

public void put(K key, V value) {      // 리스트에 추가
    if (!contains(key))     N++;
    st[hash(key)].put(key, value);
}

public void delete(K key) {              // 리스트에서 삭제
    if (!contains(key))     return;
    st[hash(key)].delete(key);  N--;
}

public Iterable<K> keys() {              // 각 리스트의 원소들을 모두 포함
    ArrayList<K> keyList = new ArrayList<K>(N);
    for (int i = 0; i < M; i++)
        for (K key : st[i].keys())
            keyList.add(key);
    return keyList;
}
```

3.2 Separate chaining의 분석

- 균등 해싱을 가정하면 각 리스트의 길이는 N / M에 비례
  - 하나의 리스트에 k개의 원소가 할당될 확률

$$\binom{N}{k}\left(\frac{1}{M}\right)^{k}\left(\frac{M-1}{M}\right)^{N-k}$$



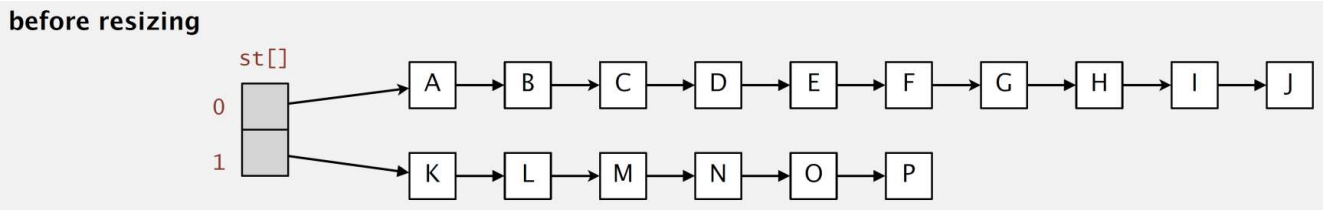
Binomial distribution ($N = 10^4$, $M = 10^3$, $\alpha = 10$)

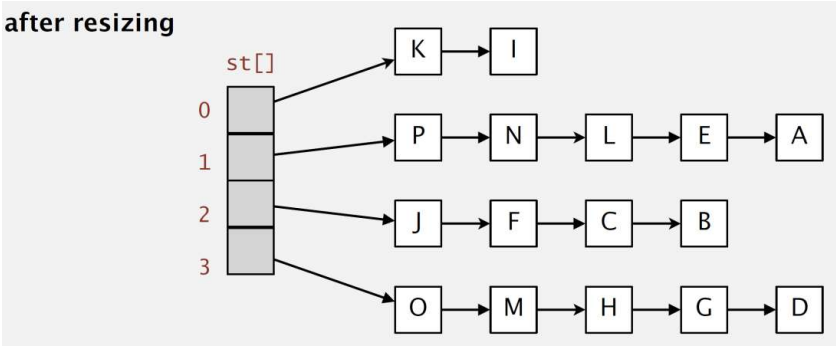
$\alpha = N / M$

- M이 너무 클 경우 → 빈 리스트의 수가 증가
- M이 너무 작을 경우 → 리스트의 길이가 증가
- M의 일반적인 선택: N / 4

해시 테이블의 재구성

- 목표: 각 리스트의 길이를 적정한 수준으로 유지
  - N / M ≥ 8 이면, 배열의 크기 M을 두 배로 증가
  - N / M ≤ 2 이면, 배열의 크기 M을 반으로 감소
  - 테이블의 재구성시 모든 원소들을 <u>다시 해싱</u>할 것!



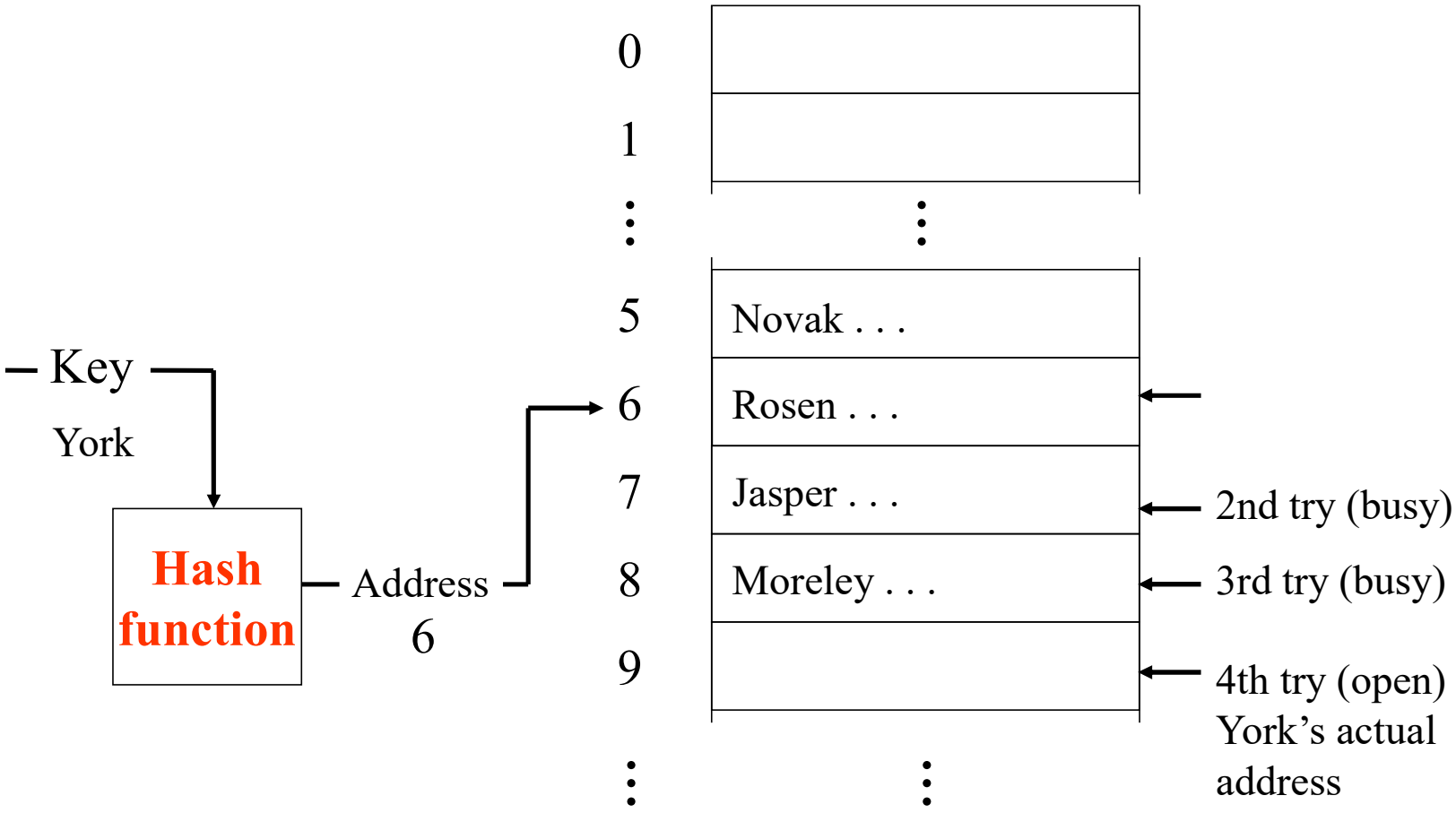


4. Linear Probing

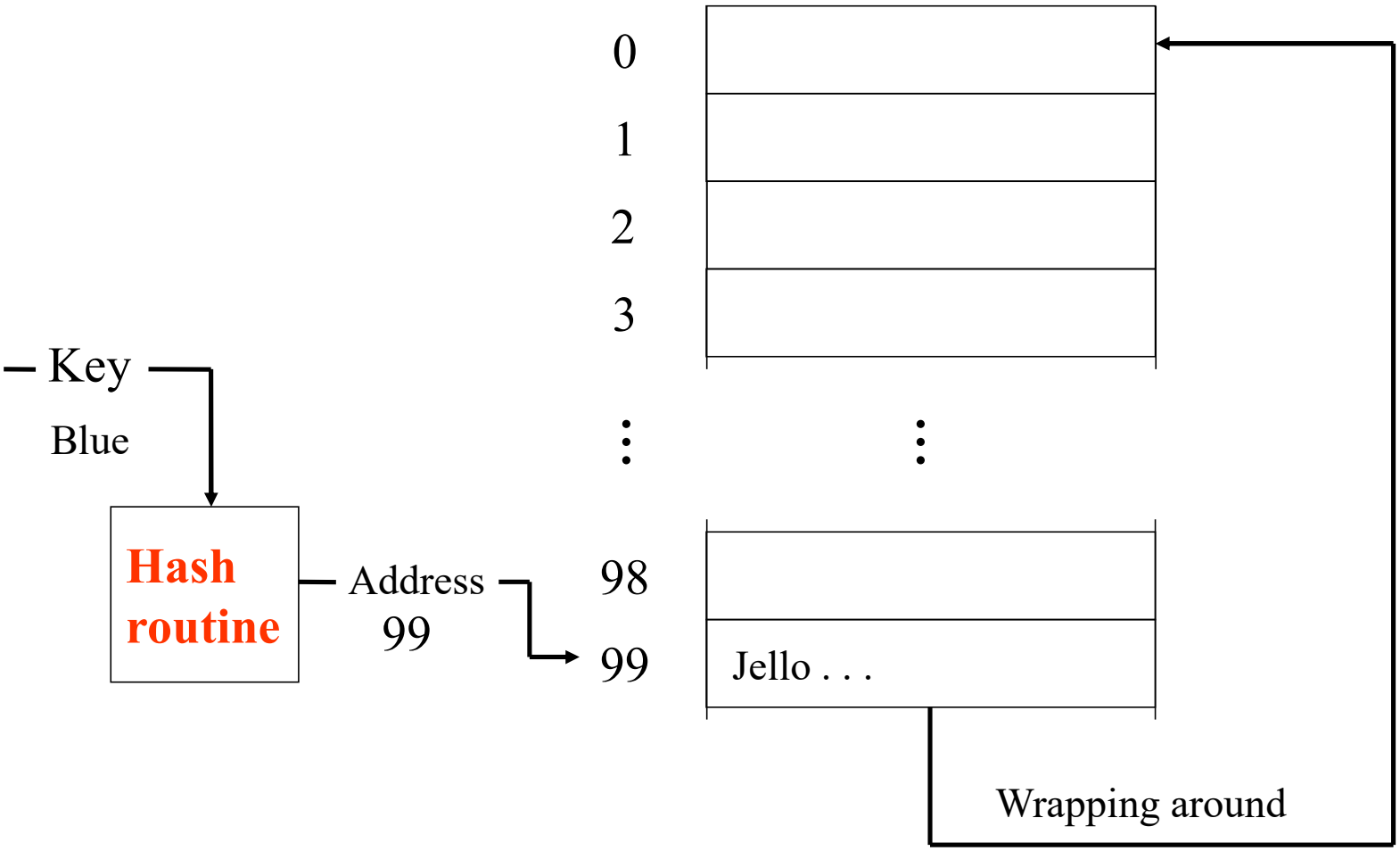
- 개방 주소법(open addressing)
  - 테이블의 크기(M) > 원소의 수(N)을 유지하며, 빈 공간을 collision 해결에 사용
  - Linear probing은 개방 주소법의 가장 단순한 형태

- 선형 탐색법(linear probing)
  - h(key) → 0 부터 M – 1 사이의 주소 i를 반환
  - get(key)의 구현
    - i에 찾는 key가 존재할 경우: search hit
    - i가 null이면: search miss
    - i에 다른 key가 저장되어 있으면: collision 발생
      - (i + 1) % M부터 이후 주소를 계속 검사
      - 언제까지?

Linear Probing의 예(1)



Linear Probing의 예(2)



Linear Probing의 예(3)

key	hash	value	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
S	6	0							S 0									
E	10	1							S 0				E 1					
A	4	2					A 2		S 0				E 1					
R	14	3					A 2		S 0				E 1				R 3	
C	5	4					A 2	C 5	S 0				E 1				R 3	
H	4	5					A 2	C 5	S 0	H 5			E 1				R 3	
E	10	6					A 2	C 5	S 0	H 5			E 6				R 3	
X	15	7					A 2	C 5	S 0	H 5			E 6				R 3	X 7
A	4	8					A 8	C 5	S 0	H 5			E 6				R 3	X 7
M	1	9		M 9			A 8	C 5	S 0	H 5			E 6				R 3	X 7
P	14	10	P 10	M 9			A 8	C 5	S 0	H 5			E 6				R 3	X 7
L	6	11	P 10	M 9			A 8	C 5	S 0	H 5	L 11		E 6				R 3	X 7
E	10	12	P 10	M 9			A 8	C 5	S 0	H 5	L 11		E 12				R 3	X 7

entries in red are new

entries in gray are untouched

keys in black are probes

probe sequence wraps to 0

keys[]
vals[]

Trace of linear-probing ST implementation for standard indexing client

4.1 Java 구현 – LinearProbingHashST

```
public class LinearProbingHashST<K,V> {
    private int N;
    private int M;          // 항상 M > N을 만족하여야 함.
    private K[] keys;       // 키를 저장하는 배열
    private V[] vals;       // 값을 저장하는 배열

    public LinearProbingHashST() {   this(31); }
    public LinearProbingHashST(int M) {
        keys = (K[]) new Object[M];
        vals = (V[]) new Object[M];
        this.M = M;
    }

    public boolean contains(K key) { return get(key) != null; }
    public boolean isEmpty() { return N == 0; }
    public int size() { return N; }

    private int hash(K key) {    return (key.hashCode() & 0x7fffffff) % M;     }
```

LinearProbingHashST 클래스(2)

```
public V get(K key) {
    for (int i = hash(key); keys[i] != null; i = (i + 1) % M)     // 테이블 순회
        if (keys[i].equals(key))       // search hit
            return vals[i];
    return null;           // search miss
}

public void put(K key, V value) {
    if (N >= M / 2)    resize(2 * M + 1);        // 테이블 재구성: 다음에 설명
    int i;
    for (i = hash(key); keys[i] != null; i = (i + 1) % M)
        if (keys[i].equals(key)) {        // 기존에 존재하는 키
            vals[i] = value;
            return;
        }
    keys[i] = key;           // 새로 (키, 값)의 쌍을 추가
    vals[i] = value;
    N++;
}
```

4.2 Linear Probing의 성능

- 적재 밀도(Packing density)

$$\text{Packing Density} = \frac{\text{입력된원소의수}}{\text{테이블의크기}} = \frac{N}{M}$$

- 주의 사항
  - Packing density와 collision ratio의 관계
  - Packing density와 space utilization의 관계

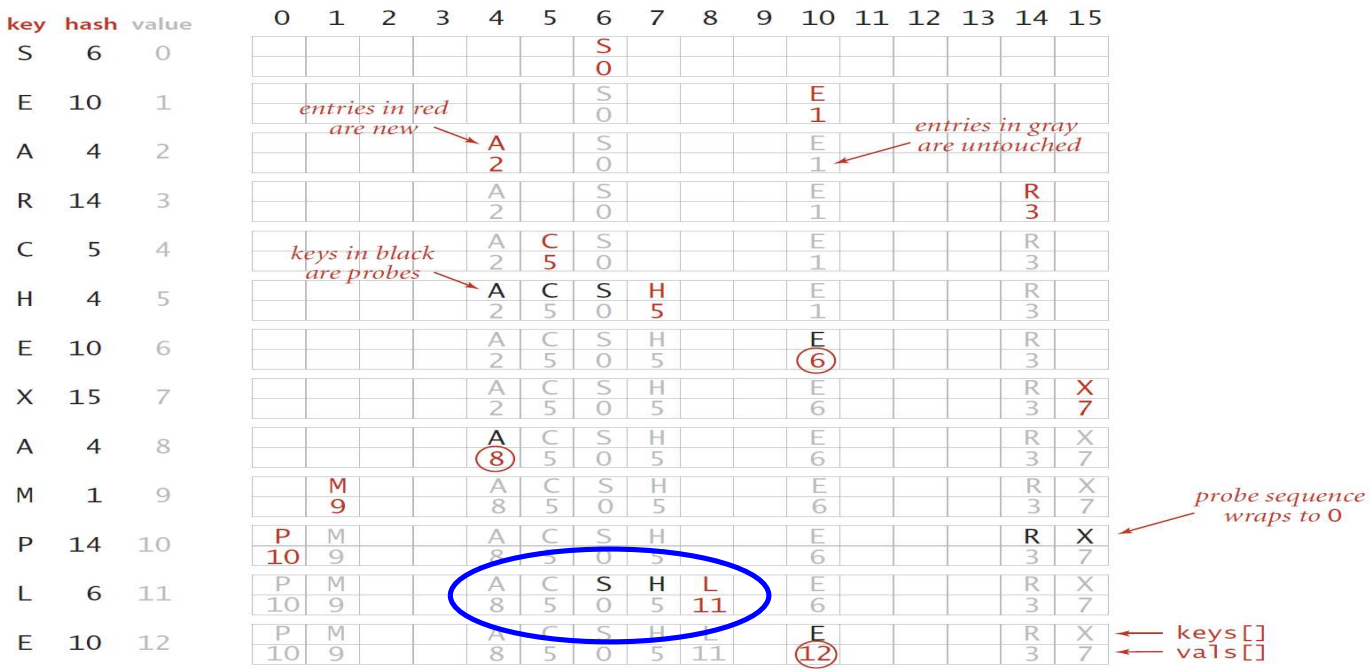
Effect of Packing Density

- Packing density별로 home address에 저장되지 못하는 레코드의 비율

Packing Density (%)	Synonyms as % of Records
10	4.8
20	9.4
30	13.6
40	17.6
50	21.4
60	24.8
70	28.1
80	31.2
90	34.1
100	36.8

Linear Probing의 문제점: Clustering

- Cluster: A contiguous block of items.
- Observation:
  - New keys likely to hash into middle of big clusters



Trace of linear-probing ST implementation for standard indexing client

Search Length

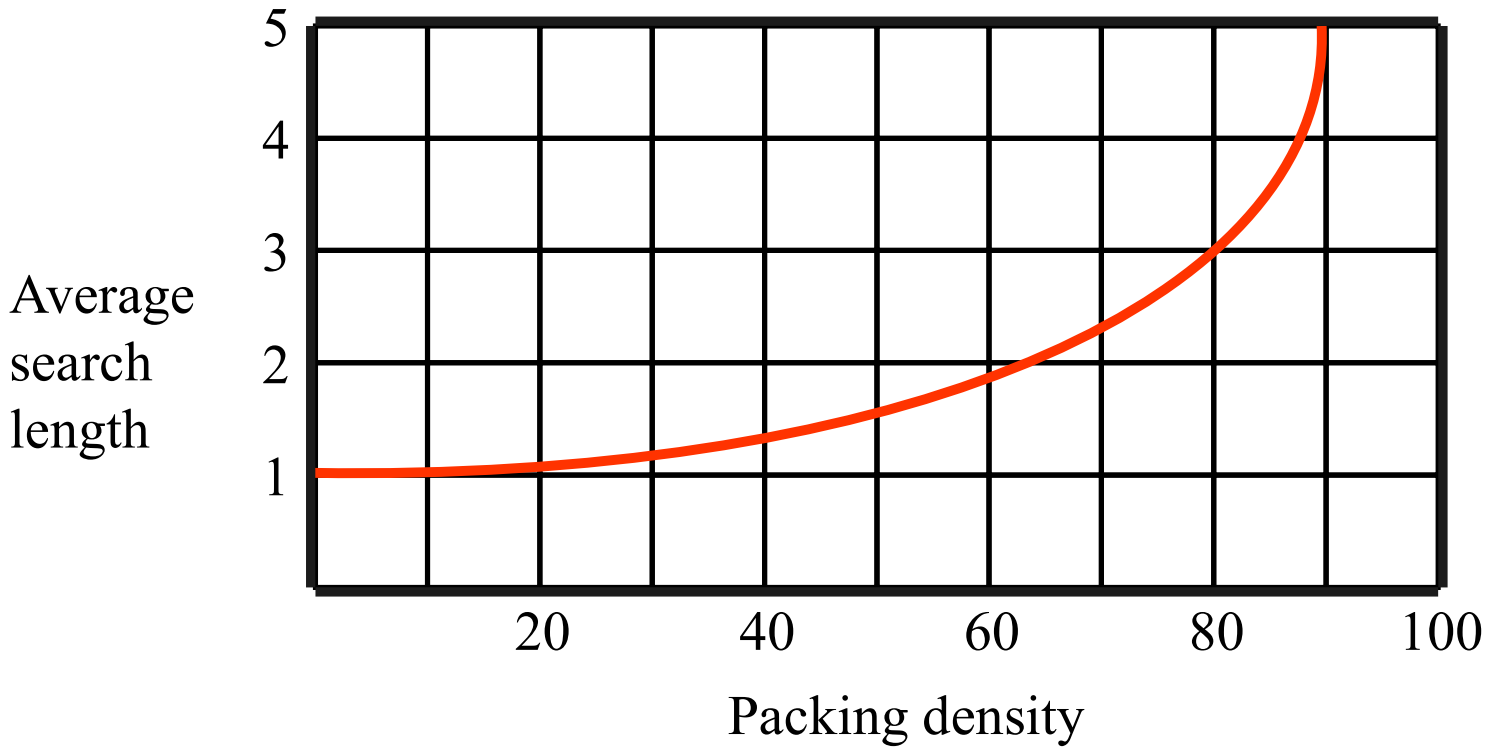
- 정의
  - 주어진 Key를 검색하기 위해 필요한 액세스 수
  - average search length = $\dfrac{\text{total search length}}{\text{total number of keys}}$
- Packing density와의 관계

Search length의 예

Key	Home Address
Adams	20
Bates	21
Cole	21
Dean	22
Evans	20

Actual address		Home address	Number of accesses needed to retrieve
0			
⋮	⋮		
20	Adams . . .	20	1
21	Bates . . .	21	1
22	Cole . . .	21	2
23	Dean . . .	22	2
24	Evans . . .	20	5
25			

Packing Density와 Search Length



해시 테이블의 재구성

- 목표: Packing density를 적정한 수준으로 유지
  - N / M ≥ 50% 이면, 배열의 크기 M을 두 배로 증가
  - N / M ≤ 12.5% 이면, 배열의 크기 M을 반으로 감소
  - 테이블의 재구성시 모든 원소들을 다시 해싱할 것!

```
private void resize(int cap) {
  LinearProbingHashST<K,V> t;
  t = new LinearProbingHashST<>(cap);
  for (int i = 0; i < M; i++)   // 새로운 테이블에 모든 원소 추가
    if (keys[i] != null)
      t.put(keys[i], vals[i]);
  keys = t.keys;
  vals = t.vals;
  M = t.M;
}
```

원소의 rehashing없이 테이블을 재구성하는 방법? → 동적 해싱

4.3 버킷(Bucket)

- 하나의 주소 공간 = 하나의 Bucket
  - 해싱의 결과로 bucket 주소 결정
  - 설계시 고려사항
    - 하나의 bucket에 몇 개의 원소(B)를 저장할 것인가?
    - Synonym들을 하나의 bucket에 저장
  - 예: 100개의 메모리가 가용
    - 방안 1: M = 100, B = 1
    - 방안 2: M = 50, B = 2
    - 방안 3: M = 25, B = 4
    - …
  - Bucket overflow?
  - Bucket내에서 검색 방법?

Bucket 크기 = 3인 경우의 예

Key	Home Address
Green	30
Hall	30
Jenks	32
King	33
Land	33
Marx	33
Nutt	33

Bucket address	Bucket contents		
30	Green . . .	Hall . . .	
31			
32	Jenks . . .		
33	King . . .	Land . . .	Marx . . .

(Nutt . . . Is an overflow record)

Linear Probing에서 다양한 Packing density와 Bucket 크기에 따른 Average search length

Packing Density (%)	Bucket Size				
	1	2	5	10	50
10	1.06	1.01	1.00	1.00	1.00
30	1.21	1.06	1.00	1.00	1.00
40	1.33	1.10	1.01	1.00	1.00
50	1.50	1.18	1.03	1.00	1.00
60	1.75	1.29	1.07	1.01	1.00
70	2.17	1.49	1.14	1.04	1.00
80	3.00	1.90	1.29	1.11	1.01
90	5.50	3.15	1.78	1.35	1.04
95	10.50	5.6	2.7	1.8	1.1

Adapted from Donald Knuth, *The Art of Computer Programming*, Vol. 3, ©1973,

4.4 Linear Probing에서 삭제 연산

- Record 삭제 시 고려 사항
  - Collision으로 인해 record가 다른 위치에 존재 가능
    - Record 삭제 후 재배치 필요
  - Free slot은 이후 새로운 record에 할당 가능하도록.

record	Home address	Actual address
Adams	5	5
Jones	6	6
Morris	6	7
Smith	5	8

	. . .
4	
5	Adams . . .
6	Jones . . .
7	~~Morris~~ . . .
8	Smith . . .
	. . .

두 가지 접근 방법

방법 1: Morris만 삭제하고, 나머지는 그대로 두자.

	. . .
4	
5	Adams . . .
6	Jones . . .
7	
8	Smith . . .
	. . .

방법 2: Morris의 자리에 tombstone을 두자.

	. . .
5	Adams . . .
6	Jones . . .
7	# # # # # #
8	Smith . . .
	. . .

Lazy Deletion - Tombstone

- 기본 개념
  - Record 삭제 후, 삭제 되었다는 표시를 남기자.
  - Free slot과는 구분

- 장점
  - Search 중 tombstone이 발견되면, 계속 search
  - Tombstone 위치에 새로운 record 저장 가능

- 주의 사항
  - Searching delay (앞 그림에서 Smith 삭제 시)
  - 중복 확인 곤란 (앞 그림에서 새로운 Smith 삽입)

Lazy Deletion의 문제점

- 거듭되는 deletion의 결과로, tombstone 과다 존재
  - searching delay 초래

- 해결 방안
  - Local reorganization at each deletion
  - Complete reorganization after threshold
  - 다른 collision resolution algorithm 사용

Local Reorganization

Key	Home Address
Adams	20
Bates	21
Cole	21
Dean	22
Evans	22
Frank	24

Address	Key
20	Adams
21	~~Bates~~
22	Cole
23	Dean
24	Evans
25	Frank

Bates 삭제

Address	Key
20	Adams
21	Cole
22	
23	Dean
24	Evans
25	Frank

Evans 이동

Address	Key
20	Adams
21	Cole
22	Evans
23	Dean
24	
25	Frank

Frank 이동

Address	Key
20	Adams
21	Cole
22	Evans
23	Dean
24	Frank
25	

LinearProbingHashST 클래스(3)

```
public void delete(K key) {
    if (!contains(key)) return;

    int i = hash(key);     // 테이블에서 key가 저장된 위치를 검색
    while (!key.equals(keys[i]))
        i = (i + 1) % M;

    keys[i] = null;  vals[i] = null;   // 키와 값을 삭제

    i = (i + 1) % M;                   // rehash all keys in same cluster
    while (keys[i] != null) {          // delete keys[i] and vals[i], and reinsert
        K keyToRehash = keys[i];
        V valToRehash = vals[i];
        keys[i] = null; vals[i] = null; N--;
        put(keyToRehash, valToRehash);
        i = (i + 1) % M;
    }
    N--;
}
```

교과서에 나온
delete() 알고리즘

5. Other Collision Resolution Techniques

- Two-probe Hashing (Separate Chaining을 개선)
  - 두 개의 해시 함수를 이용
  - 길이가 짧은 리스트에 추가

- Quadratic Probing (Linear probing을 개선)
  - 충돌 발생시 보폭을 제곱으로 넓힌다
    - h(x)+1, h(x) + 4, h(x) + 9, h(x) + 16, …
  - Cluster에서 빨리 탈출 가능
  - 그러나 h(a) = h(b)일 경우에는 a와 b가 동일한 주소에 계속 mapping ← Secondary Clustering

- Double Hashing (Linear probing을 개선)
- Cuckoo Hashing (Linear probing을 개선)

Double Hashing

- 두 개의 해시 함수(h와 f)를 이용하여 해시 함수를 생성
  - i번째 충돌시 다음 주소: $h_i(x) = (h(x) + i*f(x)) \% m$
  - h(k1) == h(k2) && f(k1) == f(k2)일 확률이 매우 낮다
  - 그 결과로 Cluster의 발생을 방지

- 두 개의 해시 함수를 만들 때 권장하는 방법
  - h(x) = hashCode(x) % m
  - f(x) = 1 + (hashCode(x) % m')
  - m'는 m보다 조금 작은 소수이며, m과 m'는 서로 소
  - 예: (m = 13, m' = 11) 혹은 (m = 1601, m' = 1597)

- 단점: Deletion을 구현하기가 쉽지 않다.

Quadratic Probing과 Double Hashing의 예

Quadratic Probing

$h_i(x) = (h(x) + i^2) \bmod 13$

예: 입력 순서 15, 18, 43, 37, 45, **30**

0	
1	
2	15
3	
4	43
5	18
6	45
7	
8	30
9	
10	
11	37
12	

Double Hashing

$h(x) = x \bmod 13, f(x) = (x \bmod 11) + 1$

$h_i(x) = (h(x)+if(x)) \bmod 13$

예: 입력 순서 15, 19, **28**, **41**, **67**

0		
1		
2	15	$h_0(15) = h_0(28) = h_0(41) = h_0(67) = 2$
3		
4	67	$h_1(67) = 4$
5		
6	19	
7		
8		
9	28	$h_1(28) = 9$
10		
11	41	$h_1(41) = 11$
12		

Cuckoo Hashing(1)

- Linear probing에서 최대 search length를 2로 제한
- 해시 테이블의 구조
  - Linear probing의 해시 테이블 크기 = M이라면,
  - M을 크기가 M/2인 두 개의 테이블(T1, T2)로 분할
    - 각 테이블의 해시 함수: h1과 h2
    - 모든 키는 T1 또는 T2 중의 하나에만 저장

- get(k)
  - T1[h1(k)].key == k이면 T1[h1(k)].value 반환
  - T2[h2(k)].key == k이면 T2[h2(k)].value 반환

Cuckoo Hashing(2)

- put(k, v)
  - T1에서 h1(k)에 k가 저장되어 있는지 확인
  - 없으면, T1[h1(k)] 에 (k, v)를 저장하고,
  - 원래 T1[h1(k)]에 있던 (k', v')는
    - T2[h2(k')]에 저장 ← 남의 둥지를 뺏는다

```
procedure insert(x)
   if lookup(x) then return
   loop MaxLoop times
      if T1[h1(x)] = ⊥ then { T1[h1(x)] ← x; return }
      x ↔ T1[h1(x)]
      if T2[h2(x)] = ⊥ then { T2[h2(x)] ← x; return }
      x ↔ T2[h2(x)]
   end loop
   rehash(); insert(x)
end
```

Cuckoo Hashing의 동작 과정

k	h1	h2
20	9	1
50	6	4
53	9	4
75	9	6
100	1	9
67	1	6
105	6	9
3	3	0
36	3	3
39	6	3

T1 테이블 (해시 함수 = h1)										
	20	50	53	75	100	67	105	3	36	39
0										
1					100	67	67	67	67	100
2										
3								3	36	36
4										
5										
6		50	50	50	50	50	105	105	105	50
7										
8										
9	20	20	53	75	75	75	53	53	53	75

T2 테이블 (해시 함수 = h2)										
	20	50	53	75	100	67	105	3	36	39
0									3	3
1			20	20	20	20	20	20	20	20
2										
3										39
4				53	53	53	50	50	50	53
5										
6							75	75	75	67
7										
8										
9						100	100	100	100	105

Cuckoo Hashing의 고려 사항

- 조건: 각 테이블의 크기(r) ≥ (1+ε)*입력 데이터 수(N)
  - 즉, 적재 밀도(Packing density)는 0.5 미만
  - 권장하는 적재 밀도는 0.3 정도

- 적재 밀도가 커짐에 따라 삽입 연산에서 반복문의 길이가 길어진다.
  - 경우에 따라서는 무한 루프도 발생 가능
  - 반복 횟수가 MaxLoop보다 커질 경우, 테이블의 크기를 2배로 증가하고 (key, value) 쌍들을 재배치
  - MaxLoop의 설정: $3 * \log_{1+\varepsilon} N$

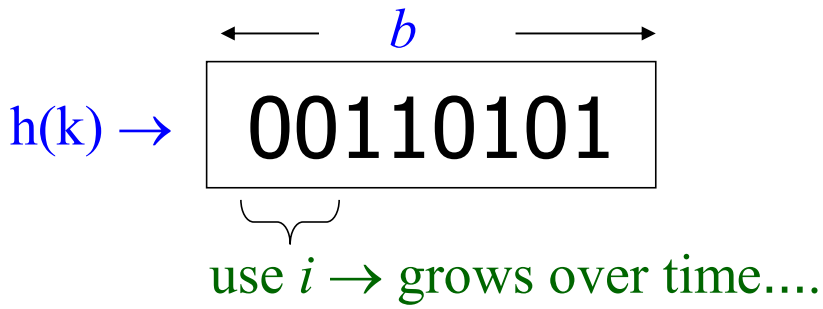
6. Dynamic Hashing

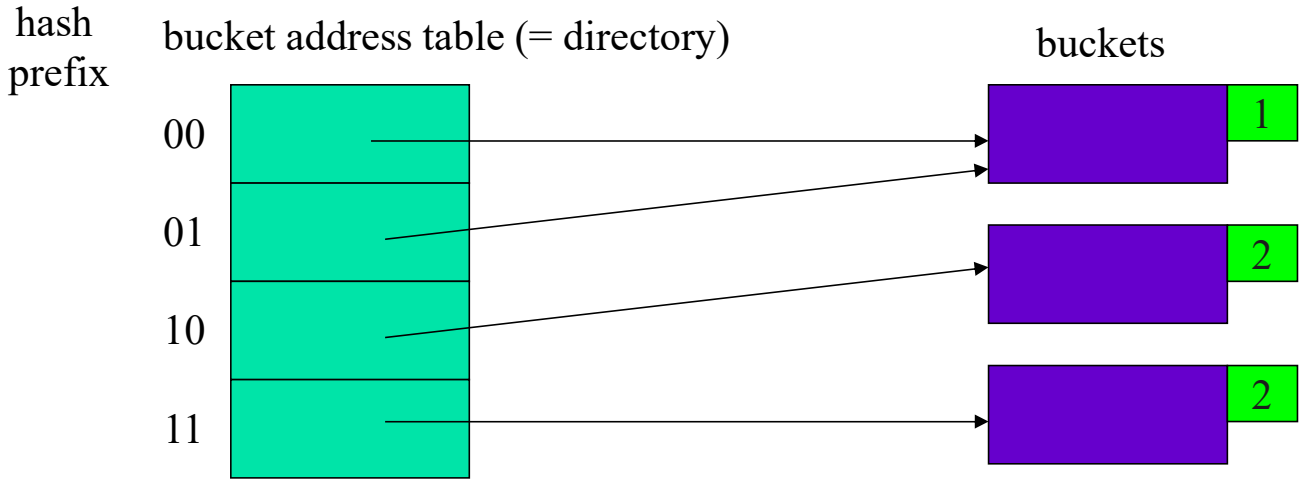
- 데이터 수가 증가할 경우 Static Hashing
  - 현재 테이블 크기를 기준으로 한 hash function 설계
  - 예상 테이블 크기를 기준으로 한 hash function 설계
  - Hash function을 주기적으로 재 계산

- Dynamic Hashing
  - Dynamic directory size
  - Controlled overflow/underflow
  - Extensible hashing (Directory를 이용한 hashing)
  - Linear Hashing (Directory가 없는 hashing)

Extensible Hashing

- 기본 개념
  - Use *i* of *b* bits output by hash function.





Example

branch-name	h(branch-name)
Round Hill	0101 1000 0011 1111 1001 1100 0000 0001
Downtown	1010 0011 1010 0000 1100 0110 1001 1111
Perryridge	1111 0001 0010 0100 1001 0011 0110 1101
Redwood	1011 0101 1010 0110 1100 1001 1110 1011
Clearview	1101 0101 1101 1110 0100 0110 1001 0011
Brighton	0010 1101 1111 1011 0010 1100 0011 0000
Mianus	1000 0111 1110 1101 1011 1111 0011 1010

Initial Extensible Hash Structure ($i = 0$)

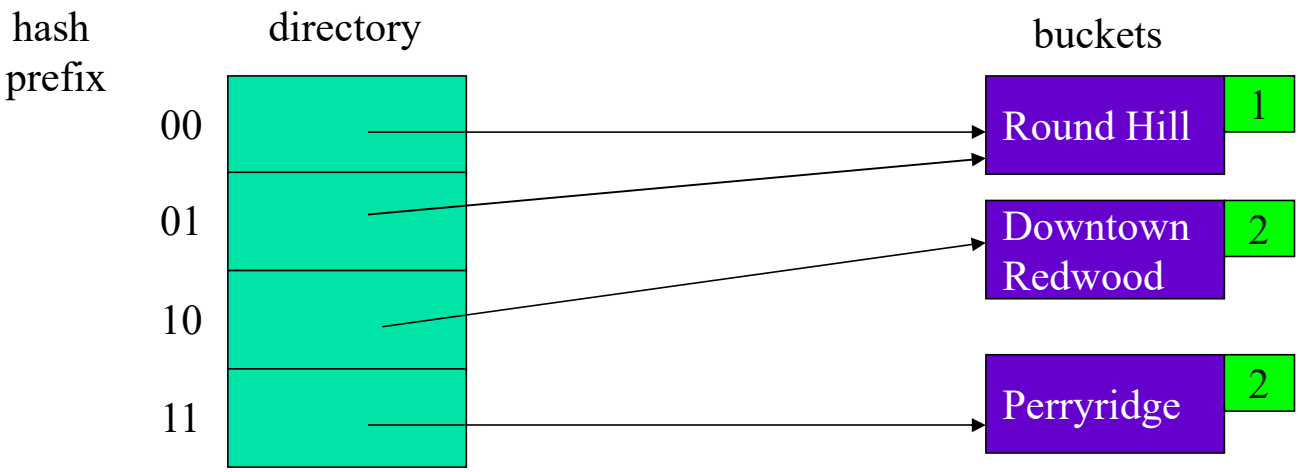
Example (Cont'd)

Hash Structure after 3 Insertions ($i = 1$)

branch-name	h(branch-name)
Round Hill	0101 1000 0011 1111 1001 1100 0000 0001
Downtown	1010 0011 1010 0000 1100 0110 1001 1111
Perryridge	1111 0001 0010 0100 1001 0011 0110 1101
Redwood	1011 0101 1010 0110 1100 1001 1110 1011
Clearview	1101 0101 1101 1110 0100 0110 1001 0011
Brighton	0010 1101 1111 1011 0010 1100 0011 0000
Mianus	1000 0111 1110 1101 1011 1111 0011 1010

Example (Cont'd)

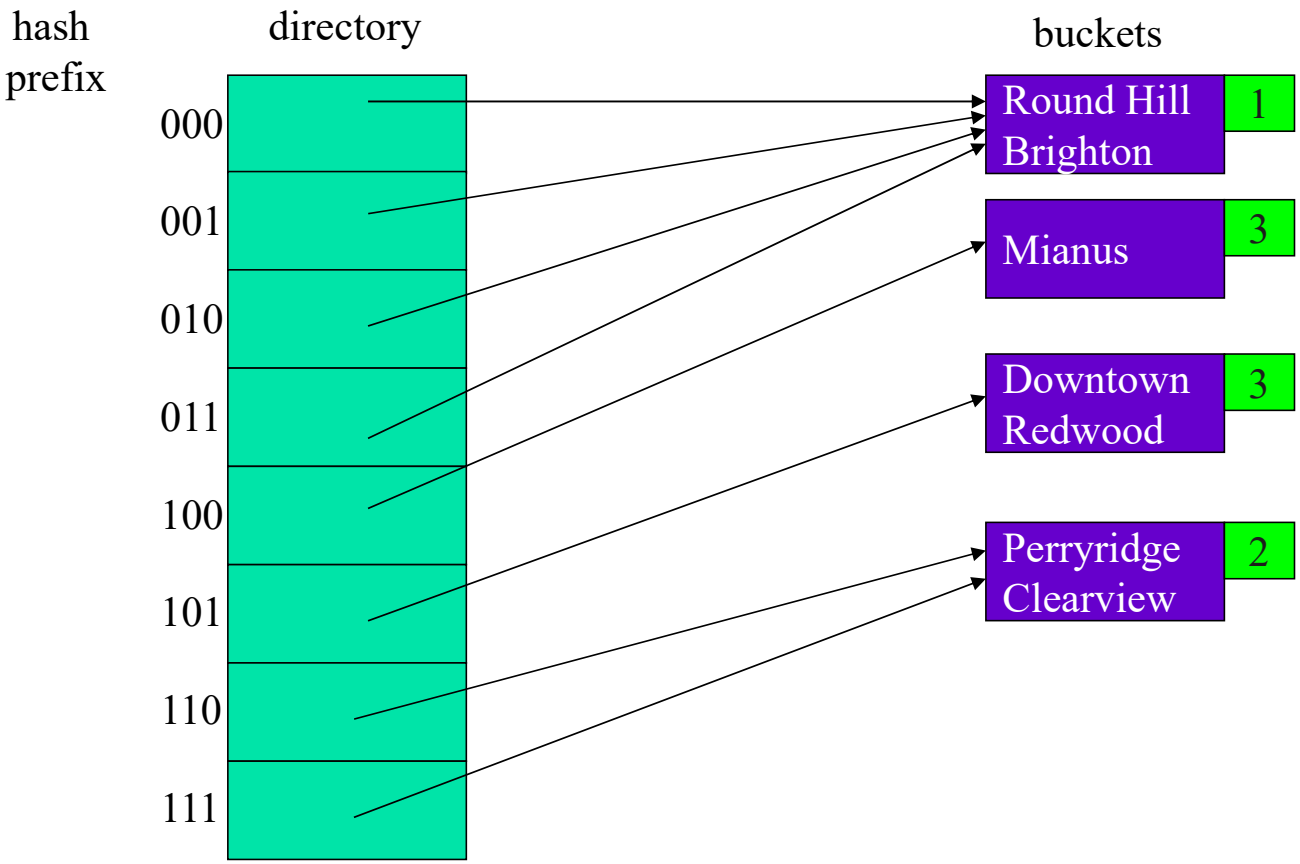
Hash Structure after 4 Insertions ($i = 2$)



+ Clearview (1101 0101 1101 1110 0100 0110 1001 0011)
+ Brighton (0010 1101 1111 1011 0010 1100 0011 0000)
+ Mianus (1000 0111 1110 1101 1011 1111 0011 1010)

Example (Cont'd)

Hash Structure after 7 Insertions ($i = 3$)



Linear Hashing

- Extensible Hashing의 단점
  - Directory의 크기가 2배로 증가할 때, 처리 시간 증가
  - 2배로 증가한 directory를 memory에 저장할 수 없을 경우, 디스크 I/O 증가
  - 특정 bucket에 레코드가 폭주할 경우, directory 크기 증가

- Linear Hashing의 개념
  - 전체 Bucket 수(**n**)에 비해 저장된 레코드 수(**r**)가 과다할 경우 Bucket 수를 <u>하나씩</u> 증가
  - Overflow 발생 가능 (**r** / **n** < threshold)

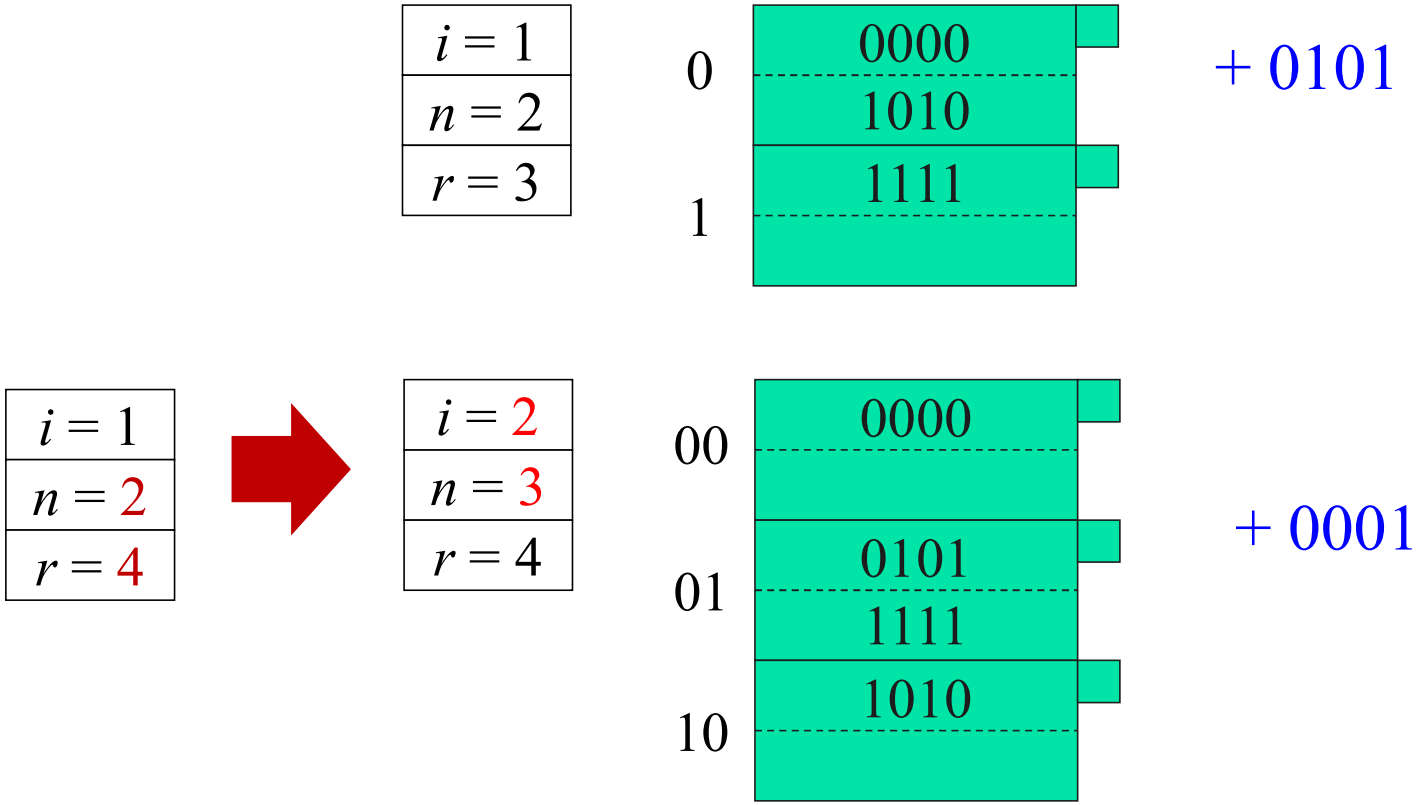
Linear Hashing의 알고리즘

- Lookup
  - h(K)의 결과에서 마지막 **i** bits를 주소로 사용
  - **n**개의 버킷[0..n-1]이 존재. 단, $2^{i-1} < n \leq 2^i$.
  - h(K)의 마지막 i bit가 m = (a^1 a^2 … a^i)라고 가정.
    - m < n이면, 버킷 m에서 레코드 검색
    - $n \leq m < 2^i$이면, (0 a^2 … a^i) 버킷에서 레코드 검색

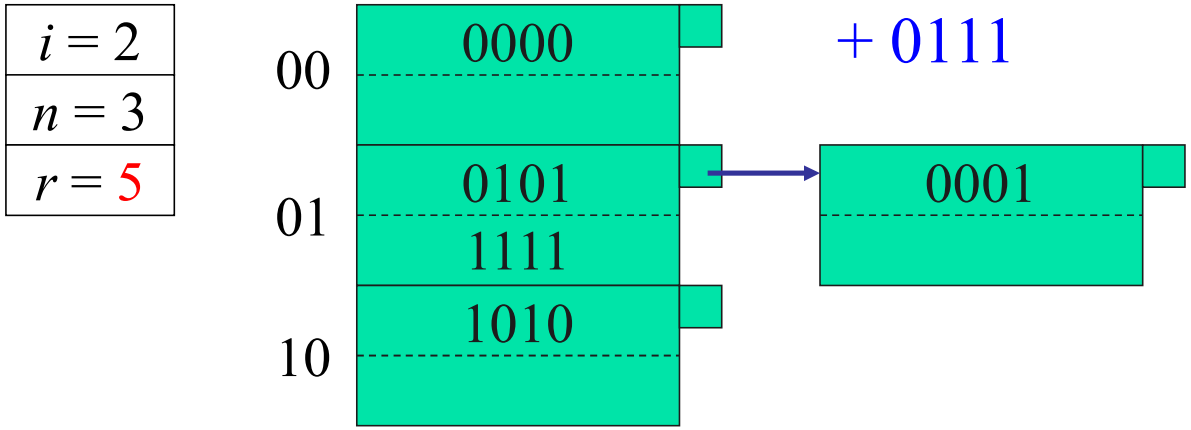
- Insert
  - Lookup의 결과 bucket B가 underflow이면, B에 저장
  - B가 full일 경우, overflow bucket을 생성하여 저장.
  - (**++r** / n > threshold)일 경우, n = n + 1
    - n > 2^i 일 경우, i = i + 1
    - (n – 1) – 2^{i-1}의 bucket을 분할하여 새 bucket에 할당

Example - Insertion

- Example (Threshold = 1.7)

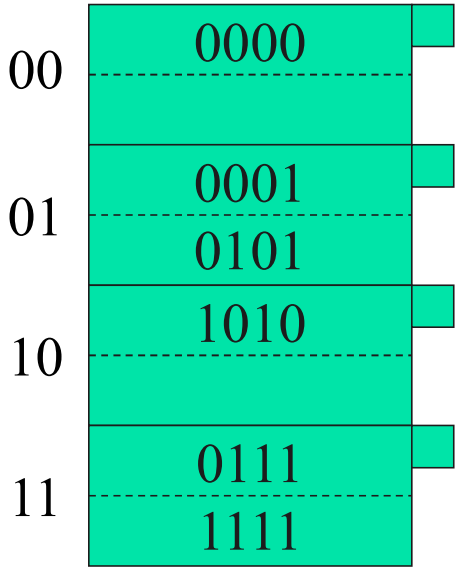


Example - Insertion



Example - Insertion

$i = 2$
$n = 4$
$r = 6$



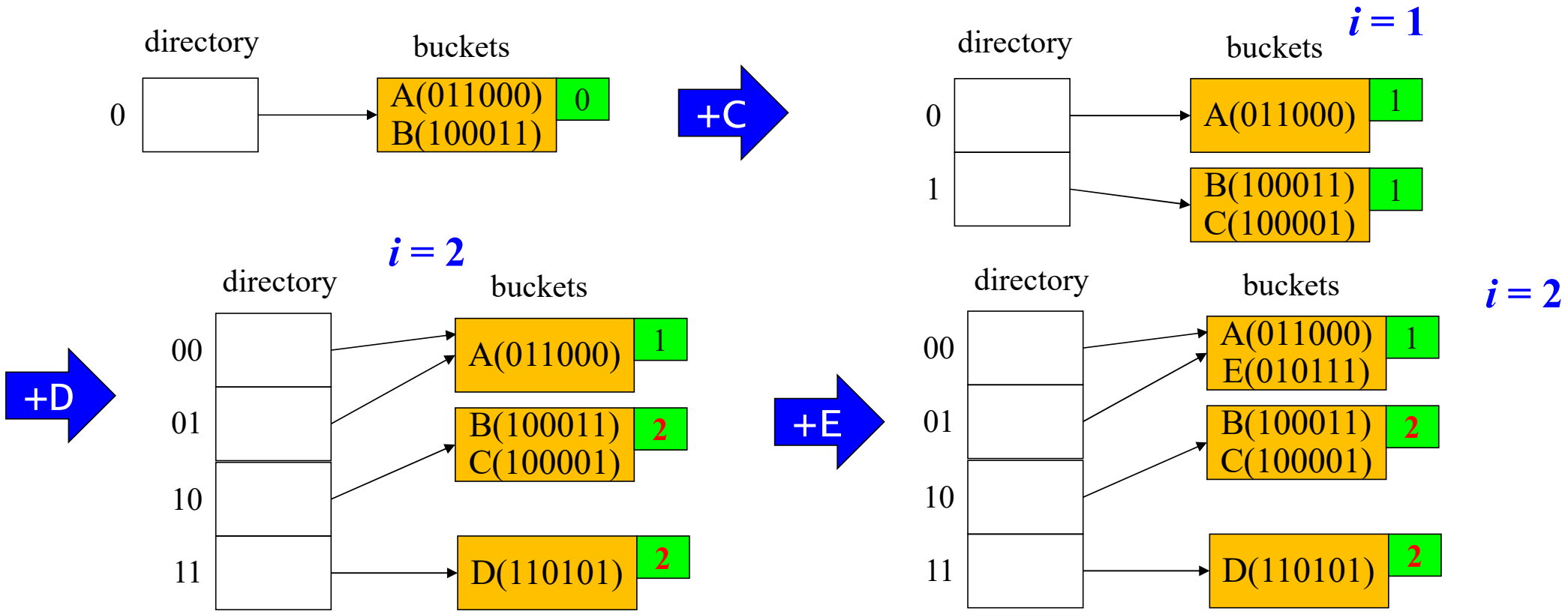
연습 문제

- A부터 H까지 8개의 key들이 차례대로 입력되고, 각 버킷에 2개의 key들이 저장될 수 있다고 가정하자. 각 key에 대한 해쉬 함수 값은 아래와 같다. Extensible Hashing과 Linear Hashing의 경우에 대해, 모든 key들이 저장되고 난 후의 해쉬 테이블을 각각 그리시오. 단, Linear Hashing에서 threshold는 1.7로 정의된다.

key	h(key)	key	h(key)	key	h(key)	key	h(key)
A	011000	B	100011	C	100001	D	110101
E	010111	F	101100	G	000111	H	111110

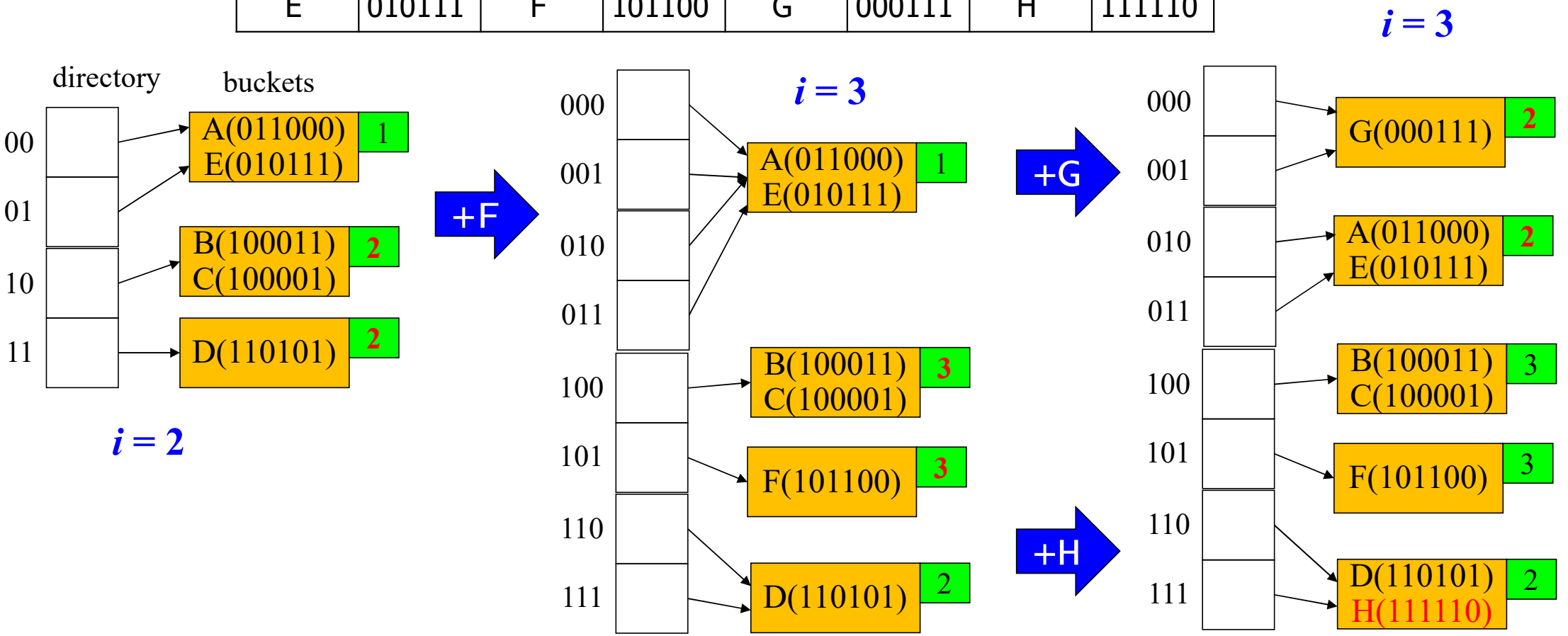
예: Extensible Hashing(1)

A	011000	B	100011	C	100001	D	110101
E	010111	F	101100	G	000111	H	111110



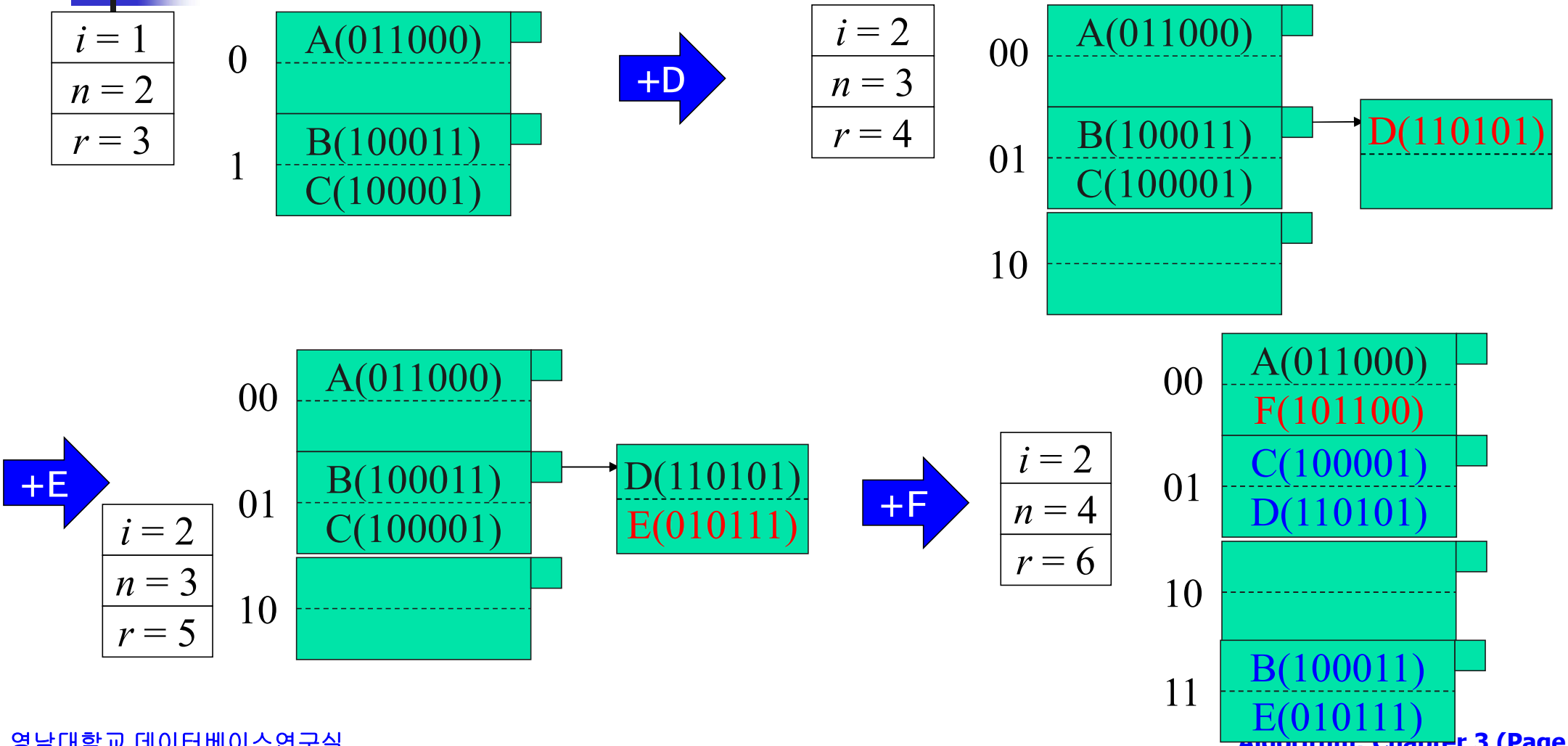
예: Extensible Hashing(2)

A	011000	B	100011	C	100001	D	110101
E	010111	F	101100	G	000111	H	111110



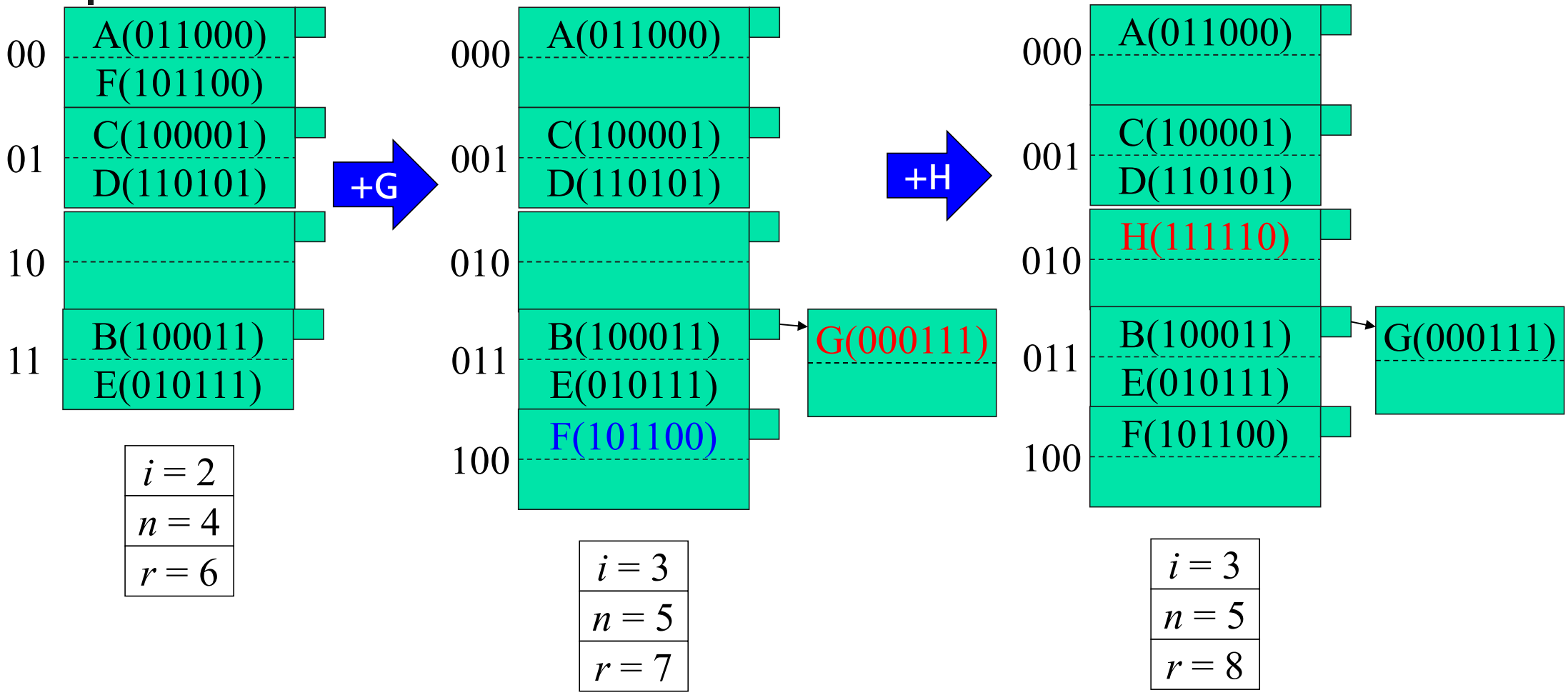
예: Linear Hashing(1)

A	011000	B	100011	C	100001	D	110101
E	010111	F	101100	G	000111	H	111110



예: Linear Hashing(2)

A	011000	B	100011	C	100001	D	110101
E	010111	F	101100	G	000111	H	111110



$i = 2$	$i = 3$	$i = 3$
$n = 4$	$n = 5$	$n = 5$
$r = 6$	$r = 7$	$r = 8$

Hash Table vs. Balanced Search Tree

- Hash tables
  - 구현이 용이
  - 키의 순서에 대한 연산은 지원 불가
  - 검색 성능이 우수(hashCode 계산 vs. log N의 트리 순회)
  - String에 대한 Java의 우수한 지원 (예: cached hash code)

- Balanced Search Tree
  - 모든 키에 대해 동일한 수준의 성능 보장
  - 순서화된 연산의 지원(구간 검색)

- Java는 두 가지 알고리즘들을 모두 지원
  - Red-black BSTs: java.util.TreeMap, java.util.TreeSet
  - Hash tables: java.util.HashMap, java.util.HashSet